Loading Data Using the COPY Command

Understand how to load transformed data into data repositories using the COPY command. Explore the syntax, options, and considerations for handling incremental and full loads to avoid duplicates and maintain data integrity.

Generally, how we load the data depends on the output of the extracted data. Let's discuss some considerations when loading data into the destination repository.

The `COPY` command

The `COPY` command is an extremely useful command for loading data. It's one of the most efficient ways to load files (from the local machine or a file storage repository like S3 or Google Bucket) to a database/data warehouse. The `COPY` command can be executed as a SQL query, a command line statement, or as a Python script, and it appends data to the existing data.

The `COPY` command is available on many database platforms, although, for some, it is called by a different name; for example, in MySQL, we load files of data by using `LOAD FROM`, and in BigQuery, we can load files using `LOAD DATA` instead of `COPY`. Read the database documentation before loading to check the loading mechanism the database vendor offers.

Ask

The general syntax for using the `COPY` command is this:

main.sql PostgreSQL

```sql
-- COPY command sytanx - All bracketed items are optional.
COPY table_name [ ( column_name [, ...] ) ]
FROM { 'filename' | PROGRAM 'command' | STDIN }
[ [ WITH ] ( option [, ...] ) ]
[ WHERE condition ]

-- "option" can be one of the following:
    FORMAT format_name
    FREEZE [ boolean ]
    DELIMITER 'delimiter_character'
    NULL 'null_string'
    HEADER [ boolean | MATCH ]
```

The COPY command syntax

For example, we can use the `COPY` command to load a CSV file named `file.csv` to an existing table called `example_table`. We'll also use the option `DELIMITER` to specify that the file is separated by commas.

main.sql PostgreSQL

```sql
COPY example_table
FROM '/PathTo/file.csv'
DELIMITER ','
CSV HEADER;
```

Ask

Loading file.csv to example_table using the COPY command

The `COPY` command options

By default, the `COPY` command inserts data into the existing table in the same order as the field in the input file. In other words, the order of the columns in the file should match the columns in the table before loading the data. If we want to manually specify the order of the columns to be loaded into the table, we can use the following syntax:

main.sql PostgreSQL

```
COPY example_table(Col1, Col2, Col3, Col4)
FROM '/PathTo/file.csv'
DELIMITER ','
CSV HEADER;
```

Loading file.csv to example_table using the COPY command

Other than making sure that the table exists and the columns are in the right order, there's an additional consideration we need to make. If the extracted data came from an incremental extract approach from a repository used for "insert-only" purposes, then we can just use the `COPY` command and be done with it. However, suppose the extracted data contains not only new records that should be appended to the destination but *updated* records as well. In that case, additional work to do in order to avoid causing duplicate records or illogical situations.

Ask

Depending on the situation, we'll either need to delete the existing records in the destination repository before loading the updated records or, if we need to keep both records, utilize a column that can differentiate between the records, such as a `last_updated_date`.

Using similar logic, when implementing a full load of the entire source repository into the destination, we also need to make sure that the destination is empty in order to not cause duplicate records. Concretely, when implementing a full load we should use the `TRUNCATE` command on the table before using the `COPY` command.

Mark As Completed

Ask